

Unit 2: The RSA Algorithm

Index

1. Principles
2. Parameters and key generation
3. Encryption and Decryption
4. Using Blocks
5. Chinese Remainder Theorem

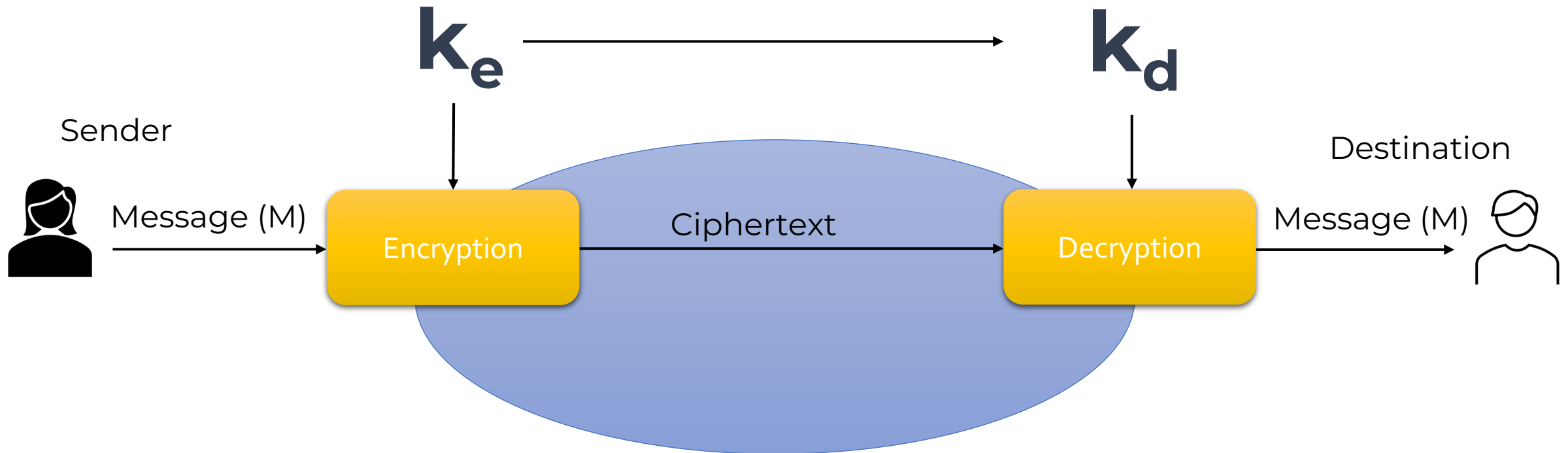
Introduction

Introduction

- RSA was created by Rivest, Shamir y Adleman in February 1978
 - 15 months after Diffie-Hellman was published
- Mentioned in 1973 by Clifford Cocks (GCHQ)
 - High computational cost for the time
 - Confidential (GCHQ is the British CIA)
- Most well-known public key algorithm
 - Elliptic curves are more efficient but they are more recent, so they are not compatible with older systems



Encryption in RSA



Parameters and key generation

Key generation in RSA

- Each user generated their own pair of public and private keys
- 1. Pick two primes p and q and calculate $n = p \cdot q$
 - n is part of the public key and we use it to establish the key length
- 2. Calculate Euler totient function : $\phi(n) = (p - 1)(q - 1)$
- 3. Calculate public key e so:
 1. $1 < e < \phi(n)$
 2. e is a prime relative with $\phi(n)$, this is $\text{lcm}[e, \phi(n)] = 1$
- 4. Calculate private key d as the multiplicative inverse of the public key in $\phi(n)$, this is $d = \text{inv}[e, \phi(n)]$
- 5. Make e, n public, keep d as a secret

Inverse values

- In public cryptography there are two keys that are inverses
- In RSA we calculate the multiplicative inverse
- This is: $\text{inv}(a, n) = b$, which means $b \cdot a \bmod n = 1$
- $\text{inv}(a, n)$ will exist if and only if $\text{lcm}(a, n) = 1$, and is unique
- If $n = 15$, $\text{inv}(8, 15)$ exists because $8 = 2 \times 2 \times 2$ y $15 = 3 \times 5$.

$$0 \times 8 \bmod 15 = 0 \quad 1 \times 8 \bmod 15 = 8 \quad \mathbf{2 \times 8 \bmod 15 = 1} \quad 3 \times 8 \bmod 15 = 9$$

$$4 \times 8 \bmod 15 = 2 \quad 5 \times 8 \bmod 15 = 10 \quad 6 \times 8 \bmod 15 = 3 \quad 7 \times 8 \bmod 15 = 11$$

$$8 \times 8 \bmod 15 = 4 \quad 9 \times 8 \bmod 15 = 12 \quad 10 \times 8 \bmod 15 = 5 \quad 11 \times 8 \bmod 15 = 13$$

$$12 \times 8 \bmod 15 = 6 \quad 13 \times 8 \bmod 15 = 14 \quad 14 \times 8 \bmod 15 = 7 \quad \mathbf{\text{inv}(8, 15) = 2}$$

- The easiest way to calculate inverses is with the Euclidean Algorithm

Euclidean Algorithm

Finding $x = \text{inv}(a, n)$ via EA

Do $(g_0, g_1, u_0, u_1, v_0, v_1, i) = (n, a, 1, 0, 0, 1, 1)$

While $g_i \neq 0$ do

○ $y_{i+1} = \text{Floor}(g_{i-1}/g_i) \rightarrow$ Nearest smaller integer

○ $g_{i+1} = g_{i-1} - y_{i+1} * g_i$

○ $u_{i+1} = u_{i-1} - y_{i+1} * u_i$

○ $v_{i+1} = v_{i-1} - y_{i+1} * v_i$

$i = i+1$

if $(v_{i-1} < 0)$

$v_{i-1} = v_{i-1} + n$

$x = v_{i-1}$

$x = \text{inv}(9, 25) = -11 + 25 = 14$

Example: find
 $x = \text{inv}(9, 25)$

i	y_i	g_i	u_i	v_i
0	-	25	1	0
1	-	9	0	1
2	2	7	1	-2
3	1	2	-1	3
4	3	1	4	-11
5	2	0	-9	25

Cálculo online animado: <https://dsri.github.io/modinverse/>

Security in RSA

- The security of RSA relies on the “Integer factorization” problem
- Given a number n , ¿can I obtain its two factors?
 - We don't have efficient algorithms to solve this problem
 - With large primes this is unfeasible with today's computers
- ¿What would happen if we are able to obtain the two factors?

Key length

- The key length should be at least 2.048 bits, with p and q being 1024 bits (we use n to get the key size)
- p and q should differ a few bits only
- The public key e is normally a small one so encrypting is less time consuming
 - We will normally use the Fermat number 4, $F_4 = 2^{2^4} + 1 = 2^{16} + 1 = 65.537$, which is a small prime number with certain proprieties that make it suitable for encryption.
- As $e = F_4$ has 17 bits, and given $e * d \bmod \phi(n) = 1$,
 - The secret key d will be a number close to $\phi(n)$; larger than 2.031 (2.048 – 17) bits for a key size of 2.048 bits.

Encryption and decryption

Encryption operation

1. Obtain public key from recipient
2. Calculate the ciphertext

$$C = M^{e_a} \pmod{n_a}$$

- M: The message we want to encrypt
- e_a and n_a the recipient's public key
- C: The ciphertext

Decryption operation

1. With our own private key
2. Obtain the original message

$$M = C^{p_a} \pmod{n_a}$$

- M: The original message
- p_a y n_a our private key
- C: The ciphertext

Example

- Ben wants to send k to Alice
 - $K = 55.967$
- Ben will encrypt
 - $C = K^{e_a} \pmod{n_a} = 55.967^{25} \pmod{66.331} = \dots = 16.667$
- Alice will decrypt
 - $K = C^{p_a} \pmod{n_a} = 16.667^{18.337} \pmod{66.331} = 55.967$

Alice

Public values

$$n_A = 66.331$$

$$e_A = 25$$

Private value

$$d_A = 18.377$$

RSA in practice

- RSA's implementation is described in an RFC
 - <https://www.rfc-editor.org/rfc/rfc8017>
- In practice we can use the OpenSSL suite to work with RSA
 - **Generate RSA key:**
`openssl genrsa -out <path/><keyname> <bits>`
 - **Obtain public key:**
`openssl rsa -in <path/><keyname> -out <path/><public> -pubout`
 - **Check content:**
`openssl rsa -in <path/><keyname> -text -out <path/><key.text>`
`openssl rsa -in <path/><public> -text -out <path/><public.text> -pubin`
 - **Encrypt:**
`openssl pkeyutl -encrypt -in <path/><message.txt>`
`-out <path/><encrypted.txt> -inkey <path/><public> -pubin`
 - **Decrypt:**
`openssl pkeyutl -decrypt -in <path/><encrypted.txt>`
`-out <path/><message.txt> -inkey <path/><key>`

Using blocks

Using blocks

- ¿What happens if we want to encrypt a message that is larger than n ?
 - Example: Encrypt $K = \text{XXX}$ with $n_A = 66.331$
- We can only encrypt numbers that are smaller than n
- If we want to encrypt something larger, we will have to split it into smaller numbers

Example

- With RSA

$$n = p * q = 919.231 * 694.201 = 638.131.079.431$$

0x 94 93 93 89 07 (5 Bytes)

- Therefore $\phi(n) = 638.129.466.000$, with $e = 7$ and $d = 364.645.409.143$

- We want to encrypt 0x 41 6d 69 67

- What should we do?

0x	41 6d 69 67	6f 73
=		
	1.097.689.447	28.531

Chinese Remainder Theorem

Motivation

- Decrypting the message sent to Alice earlier
 - $K = C^{p_a} \pmod{n_a} = 16.667^{18.337} \pmod{66.331} = 55.967$
- This exponentiation is very large and will take a lot of time
- We can use the Chinese Remainder Theorem to make simplify it
- Instead of doing the operations mod n we will do them mod p and mod q
- This can only be done by the owner of the private key

Decryption with the Chinese Remainder Theorem

$$N = \{A_p[C_p^{d_p} \pmod{p}] + A_q[C_q^{d_q} \pmod{q}]\} \pmod{n}$$

$A_p = q [\text{inv}(q, p)] = q^{p-1} \pmod{n}$ calculated at key generation

$A_q = p [\text{inv}(p, q)] = p^{q-1} \pmod{n}$ calculated at key generation

$d_p = d \pmod{p-1}$ $d_q = d \pmod{q-1}$ calculated at key generation

$C_p = C \pmod{p}$ $C_q = C \pmod{q}$ requires C

veces más rápido que si descriptamos con d en el cuerpo n.

- d_p , d_q , A_p and A_q can be precomputed during key generation
- For C_p and C_q we need to know C, but they are simple operations
- The only calculations that require more time are $(C_p^{d_p})$ and $(C_q^{d_q})$, but both are with a smaller modulo

Example

- Given: $p = 89, q = 31, n = p \cdot q = 89 \cdot 31 = 2.759, \phi(n) = 88 \cdot 30 = 2.640$
We pick $e = 29 \Rightarrow d = \text{inv}[e, \phi(n)] = \text{inv}[29, 2.640] = 2.549$
- If we want to encrypt $N = 1.995$, then:
 $C = N^e \bmod n = 1.995^{29} \bmod 2.759 = 141$
 $N = C^d \bmod n = 141^{2.549} \bmod 2.759 = \mathbf{1.995}$
- Now with the CRT : $\mathbf{N = \{A_p[C_p^{d_p} \bmod p] + A_q[C_q^{d_q} \bmod q]\} \bmod n}$
 $A_p = q^{p-1} \bmod n = 31^{88} \bmod 2.759 = \mathbf{713} \quad A_q = p^{q-1} \bmod n = 89^{30} \bmod 2.759 = \mathbf{2.047}$
 $d_p = d \bmod (p-1) = 2.549 \bmod 88 = \mathbf{85} \quad d_q = d \bmod (q-1) = 2.549 \bmod 30 = \mathbf{29}$
 $C_p = C \bmod p = 141 \bmod 89 = \mathbf{52} \quad C_q = C \bmod q = 141 \bmod 31 = \mathbf{17}$
Replacing in: $N = \{A_p[C_p^{d_p} \bmod p] + A_q[C_q^{d_q} \bmod q]\} \bmod n$
 $N = \{713[\mathbf{52^{85} \bmod 89}] + 2.047[\mathbf{17^{29} \bmod 31}]\} \bmod 2.759 \quad (\text{take time})$
 $N = \{713 \cdot 37 + 2.047 \cdot 11\} \bmod 2.759 = (26.381 + 22.517) \bmod 2.759 = \mathbf{1.995}$

Summary

Summary

- RSA is a public key cryptosystem
- To generate keys we need to
 - Pick p and q
 - Calculate the Euler indicator
 - Pick a public key (e) that is smaller and a prime relative of $\phi(n)$.
 - Calculate the inverse of e with $\phi(n)$.
- Security in RSA
 - Difficulty of factoring n
- Encryption and Decryption
 - Fast exponentiation (last unit)
 - Chinese Remainder Theorem

Questions?

